OFD 版式文档

阅读器功能演示 • OFD VIEWER SHOWCASE

本文档用于演示 OFD 阅读器在密集中文、英文、数字混排下的版面还原、断行、对齐、字形回退与渲染表现。

文档内嵌宋体子集；页面包含正文、页眉页脚、表格、两栏、代码行、字符覆盖区以及多字号彩色文字。

0 文档说明

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

1 页面模型与坐标

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体

2 文字对象与字体资源

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting keeps them aligned. Dense reports usually prefer the latter.

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

3 渲染流程与缓存

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting keeps them aligned. Dense reports usually prefer the latter.

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、Doc_0/Pages/Page_N/Content.xml 以及可选的 PublicRes.xml。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting keeps them aligned. Dense reports usually prefer the latter.

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、Doc_0/Pages/Page_N/Content.xml 以及可选的 PublicRes.xml。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting keeps them aligned. Dense reports usually prefer the latter.

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、

4 坐标变换与边界

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体

6 版面密度：8pt / 6pt / 5pt

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，

6.1 六磅正文（行距 2.9 mm）

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出

6.2 五磅注释（行距 2.4 mm）

五磅文字常用于票据附注、合同小字和表格脚注。此区域用于观察笔画粘连、抗锯齿灰度与行距压缩后的可读性。0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz。五磅文字常用于票据附注、合同小字和表格脚注。此区域用于观察笔画粘连、抗锯齿灰度与行距压缩后的可读性。0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz。五磅文字常用于票据附注、合同小字和表格脚注。此区域用于观察笔画粘连、抗锯齿灰度与行距压缩后的可读性。0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz。

7 字符覆盖与易混淆字形

ASCII 可打印字符： !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~

全角 ASCII：！＂＃＄％＆＇（）＊＋，－．／０１２３４５６７８９：；＜＝＞？＠ＡＢＣＤＥＦＧＨＩＪＫＬＭＮＯＰＱＲＳＴＵＶＷＸＹＺ［＼］＾＿｀ａｂｃｄｅｆｇｈｉｊｋｌｍｎｏｐｑｒｓｔｕｖｗｘｙｚ｛｜｝～

中文标点：、。〃〈〉《》「」『』【】〔〕〖〗〝〞〾……＿

数学与单位：±×÷≈≠≤≥∞∝∑∏∫√∠⊥∥△▲◇◆○●★☆→←↑↓℃℉‰%§¶†‡

易混淆汉字：己已巳未末土士日曰目自木本八入人人力刀千于千戊戌戌候侯拨拔辨辩账帐

数字串：0123456789 9876543210 12345678901234567890 3.141592653589793 2.718281828459045

拉丁字母：ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz

希腊字母：Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

重复汉字：密集密集密集

8 长 URL、路径与代码行

```
https://example.com/ofd/viewer?page=12&zoom=150&mode=text#section-3
file:///home/user/ofd-samples/dense-text/fixture-2026-10-21.ofd
/var/lib/ofd/archive/2026/10/dense-text-v1.ofd
user.name+ofd-test@example.com
const page = await doc.getPage(0);
const viewport = getPageViewport(page.getSize(), { dpi: 96 });
await page.render({ canvasContext: ctx, viewport });
if (glyph === undefined) { fallback.push(codepoint); }
for (const [index, ch] of Array.from(text).entries()) { pen += deltaX[index]; }
0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF
```

8.1 混排段落与网址断行

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、Doc_0/Pages/Page_N/Content.xml 以及可选的 PublicRes.xml。

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting keeps them aligned. Dense reports usually prefer the latter.

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、Doc_0/Pages/Page_N/Content.xml 以及可选的 PublicRes.xml。

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

Numbers deserve special attention: 0.00, 1,234.56, -78.90, +12.5%, 3.1415926535, 2.7182818284, 1e-9, 0x7F, 2026-10-21T16:54:23+08:00. A proportional font gives each digit a slightly different width, while a tabular figure setting

9 表格：数字右对齐与混合内容

序号	名称	数量	单价	金额	时间戳	批次号	备注
01	版式文档 A	120	36.50	4,380.00	2026-10-21 09:12:33	HT-2026-1021-001	正常
02	版式文档 B	8	1,280.00	10,240.00	2026-10-21 09:35:07	HT-2026-1021-002	加急
03	密集文本样本	256	12.80	3,276.80	2026-10-21 10:02:44	HT-2026-1021-003	测试
04	表格排版	64	88.00	5,632.00	2026-10-21 10:18:19	HT-2026-1021-004	—
05	字符覆盖	1	0.00	0.00	2026-10-21 10:40:02	HT-2026-1021-005	参考
06	字体子集	3	150.00	450.00	2026-10-21 11:05:56	HT-2026-1021-006	内嵌
07	两栏排版	12	66.60	799.20	2026-10-21 11:22:31	HT-2026-1021-007	正常
08	性能基线	99	9.99	989.01	2026-10-21 11:48:10	HT-2026-1021-008	基线
09	渲染回归	48	24.00	1,152.00	2026-10-21 12:03:27	HT-2026-1021-009	回归
10	转换验证	7	70.00	490.00	2026-10-21 12:29:45	HT-2026-1021-010	通过
11	归档样本	100	3.00	300.00	2026-10-21 12:55:08	HT-2026-1021-011	归档
12	汇总	629	—	27,410.01	2026-10-21 13:20:00	HT-2026-1021-012	合计

表 1：测试表格。金额列按右对齐处理，数量与单价使用等宽数字；时间戳和批次号包含连字符与数字，用于检查混排断行。

表格是密集文本的另一类典型场景。单元格里的数字通常右对齐，文字左对齐，表头居中；列宽需要根据内容分配，过窄会导致换行，过宽会浪费版面。金额、日期、编号等字段还要考虑等宽对齐，避免小数点错位。表格线可以用矢量路径绘制，也可以只保留横线，减少视觉噪音。当表格跨页时，表头应当重复出现，页脚则要标注续页信息。测试样本中的表格同时包含中文、英文、数字和百分比，用于检查混合内容的对齐效果。

版式文档不仅要显示正确，还要支持文字选择、复制和全文检索。文字选择依赖字符在页面上的位置与顺序，如果一行文字被拆成多个对象，选择时可能出现顺序错乱。因此生成样本时，同一行文字应尽量放在同一个文本片段中，只有在需要精确控制字距或颜色时才拆分。检索则依赖字符编码与字形的映射，内嵌子集字体必须保留完整的字符映射表，否则复制出来的文字会变成空白或乱码。测试时可以把页面文字复制到文本编辑器，检查标点、数字和英文是否完整。

密集文本页面会放大渲染器的性能问题。一页六十行、每行六十个字符，意味着一页就有三千六百次字形查找；如果每个字符都单独建对象，对象数量还会成倍增加。常见优化包括字形轮廓缓存、按字体缓存度量、延迟解析页面以及复用画布。测试时应当记录打开耗时、首页渲染耗时、连续翻页耗时和内存占用，并在不同字号、不同字体、不同对齐方式下分别取样。只有在数据稳定之后，性能对比才有意义。

不同工具生成的 OFD 在细节上存在差异。有的工具省略命名空间前缀，有的把资源放在公共资源目录，有的把页面内容拆成多层。渲染器不能假设文件一定符合某一种写法，而应当按规范宽松解析，遇到不认识的元素时跳过并记录告警。对于缺失的字体、损坏的图片、越界的坐标，也应当以可恢复的方式处理，不能直接崩溃。测试样本应当包含一些不那么规范的结构，用来验证容错能力。

版式文档常用于电子归档，要求内容在多年之后仍然可以准确呈现。为此，文件应当尽量内嵌字体和颜色描述，减少对外部环境的依赖。文字应当使用标准编码，避免私有字符集；图像应当采用通用格式，并保留足够的分辨率。归档系统还需要记录文件的完整性信息，例如摘要和签名，防止内容被篡改。测试样本可以模拟这些要求，检查阅读器在断网、缺少字体、字体被替换等条件下的表现。

版式文档把每一页视为一块固定尺寸的画布，文字、图形与图像都落在精确的坐标上。与流式文档不同，页面不会因为阅读窗口的宽窄而重新排列，因此同一份文件在不同设备上应当呈现出完全一致的版面效果。正因如此，版式文档适合承载合同、票据、公文、说明书等对格式有严格要求的材料。页面的物理尺寸通常以毫米为单位描述，而字体大小沿用磅值，二者之间的换算关系是一磅约等于零点三五二八毫米。坐标原点位于页面左上角，横轴向右为正，纵轴向下为正，这与常见的桌面排版系统一致。

文字对象是页面内容中出现频率最高的元素。一个文字对象由字体引用、字号、颜色以及一个或多个文本片段组成。每个文本片段除了字符内容之外，还可以携带逐字的水平或垂直步进值，用来描述字间距、两端对齐、竖排方向等细节。渲染器读取这些信息后，先按照字体文件中的字符映射找到对应字形，再根据步进值依次摆放，最后把轮廓填充到画布上。如果字体缺少某个字形，渲染器需要按照回退链选择替代字体，并记录一条告警，避免整页内容因为个别缺字而无法显示。

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

演示专用

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲

10 两栏与多字号混排

打开一份 OFD 文件通常要经历解包、解析文档结构、解析公共资源、解析页面内容这几个步骤。为了提高翻页速度，成熟的实现会把解析结果缓存起来，并按需加载页面。渲染一页时，引擎先结算绘制参数，再依次处理文字、图形、图像与注释对象，最后统一提交给画布。描边、填充、裁剪、透明度和混合模式都在这一步生效。如果页面中包含大量小字号文字，字形查找和路径构建会成为主要开销，因此字形轮廓缓存和字体回退缓存对性能影响很大。

每个页面对象都有自己的边界框，边界框的坐标相对于页面原点。对象内部还可以嵌套变换矩阵，用于旋转、缩放、镜像或平移。文字对象内部的文本片段坐标相对于对象边界框，而字形轮廓又相对于自身字体坐标系。多层变换叠加时，顺序不能颠倒，否则会出现位置偏移或方向反转。对称、旋转九十度、负坐标边界等情形在实际文件中并不罕见，渲染器需要对它们逐一处理。测试样本应当覆盖这些边界情况，并给出可以复核的页面尺寸与内容位置。

OFD 允许把字体文件一并打包，也允许只声明字体名称、由阅读器在本机查找。内嵌字体可以保证转换和打印结果稳定，但会显著增加文件体积；非内嵌字体体积小，却依赖运行环境。为了兼顾两者，常见做法是对内嵌字体做子集化，只保留文档实际使用的字形。子集化之后的字体仍然保留字符映射表，阅读器可以正常定位字形。需要注意的是，某些转换工具对子集字体的名称表和度量表比较敏感，测试时应当同时验证渲染、复制和转 PDF 三条链路。

密集文本测试关注的是单位面积内文字的数量与排布。常见参数包括字号、行距、字距、页边距、分栏数量和表格密度。八磅正文配三点六毫米行距，在 A4 纸上每页可以排下六十余行；六磅文字配二点九毫米行距，则可以排下八十余行。字号越小，字形轮廓的细节越容易被抗锯齿吞掉，笔画之间的间隙也越窄。渲染器和转换器都需要在小字号下保持字形的可辨识度，不能因为取样误差把相邻笔画粘连在一起。

两端对齐是正式文档最常用的排版方式。实现时先按自然字宽把文字排入一行，再计算剩余空间，平均分配到各个字符之间。中文可以在任意字符之间断行，英文则通常以单词为单位断行，长单词或网址需要强制拆分。行首不能出现句号、逗号、右括号等收尾标点，行尾也不能出现左括号、左引号等起始标点。这些规则看似琐碎，却直接影响版面的观感。测试样本中应当同时包含纯中文、纯英文、中英混排、长网址和连续标点，以便观察断行结果。

OFD（Open Fixed-layout Document）是一种开放版式文档格式，核心目标是保证版面在不同平台上的一致性。它使用 ZIP 容器组织内容，文档结构、页面内容、资源文件分别存放在不同的 XML 条目中。一个典型的包包含 OFD.xml、Doc_0/Document.xml、Doc_0/Pages/Page_N/Content.xml 以及可选的 PublicRes.xml。

渲染器读取 TextObject 时，需要同时处理 Font、Size、Boundary、FillColor 等属性，以及 TextCode 的 X、Y、DeltaX、DeltaY。示例：Size=8.0、DeltaX=g 12 2.822，表示每个字符步进 2.822 毫米。若 DeltaX 缺失，则使用字体度量估算步进。

在实际项目中，文本常常与数字、英文缩写和符号混排，例如：合同编号 HT-2026-1021-003，金额 ¥1,234,567.89，占比 98.7%，温度 -12.5℃，误差 ±0.02 mm。两类字符的宽度不同，断行和两端对齐都要分别计算。

测试口令：请朗读下列句子 —— 红鲤鱼与绿鲤鱼与驴，黑化肥发灰会挥发，灰化肥挥发会发黑。绕口令可以检验字形相似度和渲染清晰度，也能暴露字体缺字或回退错误。

A dense text fixture stresses every stage of the text pipeline. Shaping, glyph lookup, line breaking, justification and rasterization all run once per line, and a single page may contain several thousand characters. The renderer must keep the output stable while the input grows, and it must not degrade into quadratic behavior when the same glyph appears again and again. Cache design matters more than raw speed here.

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump! Sphinx of black quartz, judge my vow. These pangrams exercise most of the Latin alphabet, including uppercase and lowercase forms, punctuation and spacing between words.

When a document is converted to PDF, the converter has to decide how to embed fonts and how to position each glyph. If the original file uses per-character deltas, the converter can reproduce the layout exactly; if it relies on font metrics, small differences in font version may shift the line width. Both behaviors are acceptable as long as the text remains legible and the page structure is preserved.

10.1 五磅密集块（行距 2.35 mm）

密集文本块用于测试最小字号下的排版稳定性。中文：版式文档、字形轮廓、字符映射、两端对齐、字体回退、抗锯齿、透明度、裁剪区域。英文：glyph outline, character map, justification, fallback, antialiasing. 数字：1024 2048 4096 8192 16384 32768 65536。密集文本块用于测试最小字号下的排版稳定性。中文：版式文档、字形轮廓、字符映射、两端对齐、字体回退、抗锯齿、透明度、裁剪区域。英文：glyph outline, character map, justification, fallback, antialiasing. 数字：1024 2048 4096 8192 16384 32768 65536。

密集文本 Dense Text 0123456789

红色文字：渲染测试 Red

蓝色文字：字形回退 Blue

绿色文字：对齐检查 Green

灰色文字：注释说明 Gray 0123456789

页脚上方保留多行 6pt 文字，用于检查页面底部的内容边界：页面尺寸、页边距、行距与文字基线之间应当保持一致。

0101010101 密 密密 密密密 密密密密 密密密密密 密密密密密密 A1B2C3D4E5F6 98765432100101010101 密 密密 密密密 密密密密 密密密密密 密密密密密密 A1B2C3D4E5F6 98765432100101010101 密 密密 密密密 密密密密 密密密密密 密密密密密密 A1B2C3D4E5F6 9876543210